

Solución - Trabajo Práctico N°2

Principios SOLID

Diagnóstico del Código Original

Principio	Problema
SRP	La clase realiza múltiples responsabilidades.
OCP	Se modifica la clase para agregar nuevos tipos.
DIP	Depende de strings concretos.
ISP	No existen interfaces.
LSP	No existe una jerarquía correcta.

El diseño original obliga a modificar la clase principal cada vez que se desea agregar un nuevo mecanismo de notificación.

Interfaz Principal

```
public interface Notificacion {  
    void enviar(String mensaje);  
}
```

Implementaciones

```
public class EmailNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando Email: " + mensaje);  
    }  
}  
  
public class SmsNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando SMS: " + mensaje);  
    }  
}  
  
public class WhatsappNotificacion implements Notificacion {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando WhatsApp: " + mensaje);  
    }  
}
```

Nuevas Extensiones

```

public class TelegramNotificacion implements Notificacion {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Enviando Telegram: " + mensaje);
    }
}

public class PushNotificacion implements Notificacion {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Enviando Push: " + mensaje);
    }
}

```

Aplicación de Dependency Inversion Principle

```

public class GestorTurnos {
    private final Notificacion notificacion;

    public GestorTurnos(Notificacion notificacion) {
        this.notificacion = notificacion;
    }

    public void registrarTurno() {
        System.out.println("Turno registrado");

        notificacion.enviar(
            "Su turno fue registrado correctamente"
        );
    }
}

```

Ejemplo de Uso

```

public class Main {
    public static void main(String[] args) {
        Notificacion notificacion =
            new EmailNotificacion();

        GestorTurnos gestor =
            new GestorTurnos(notificacion);

        gestor.registrarTurno();
    }
}

```

Beneficios Obtenidos

- Menor acoplamiento.
- Mayor extensibilidad.
- Mejor mantenibilidad.
- Facilidad de testing.
- Diseño más profesional y escalable.

Principios SOLID Aplicados

Principio	Aplicación
SRP	Cada clase posee una única responsabilidad.
OCP	Se agregan nuevas notificaciones sin modificar código existente.
LSP	Todas las implementaciones son intercambiables.
ISP	La interfaz es mínima y clara.
DIP	Las dependencias apuntan a abstracciones.